

移动应用开发实验报告

封面信息

项目	内容
实验名称	原生应用开发
实验编号	d20301035102、d20301009202
学号	2023210599
姓名	姜少威
班级	软件 232
组别	CG3
项目名称	智能健康运动记录平台
技术栈	Android (Kotlin)

实验目的

- 1. 掌握 Android Kotlin 登录页面开发
- 2. 学会 Activity 间数据传递
- 3. 掌握 SQLite 数据库操作
- 4. 完成个人中心页面设计

实验环境

平台	开发工具	语言	SDK 版本
Android	Android Studio	Kotlin	API 24+

表 1 实验环境

实验步骤

1. 需求分析

1.1 功能需求

- 用户名输入
- 密码输入
- 登录按钮
- 注册按钮
- 输入验证（不能为空、长度限制）

1.2 界面需求

- Material Design 风格（Android）
- 输入框占位提示
- 密码隐藏显示

1.3 跳转需求

- 登录成功 → 个人中心
- 点击注册 → 注册页面
- 传递用户名数据

2. Android 登录页面开发

2.1 项目创建

- 打开 Android Studio
- 创建 Empty Activity 项目
- 选择 Kotlin 语言
- 最低 SDK：API 24

2.2 系统架构设计

1 系统架构框架图

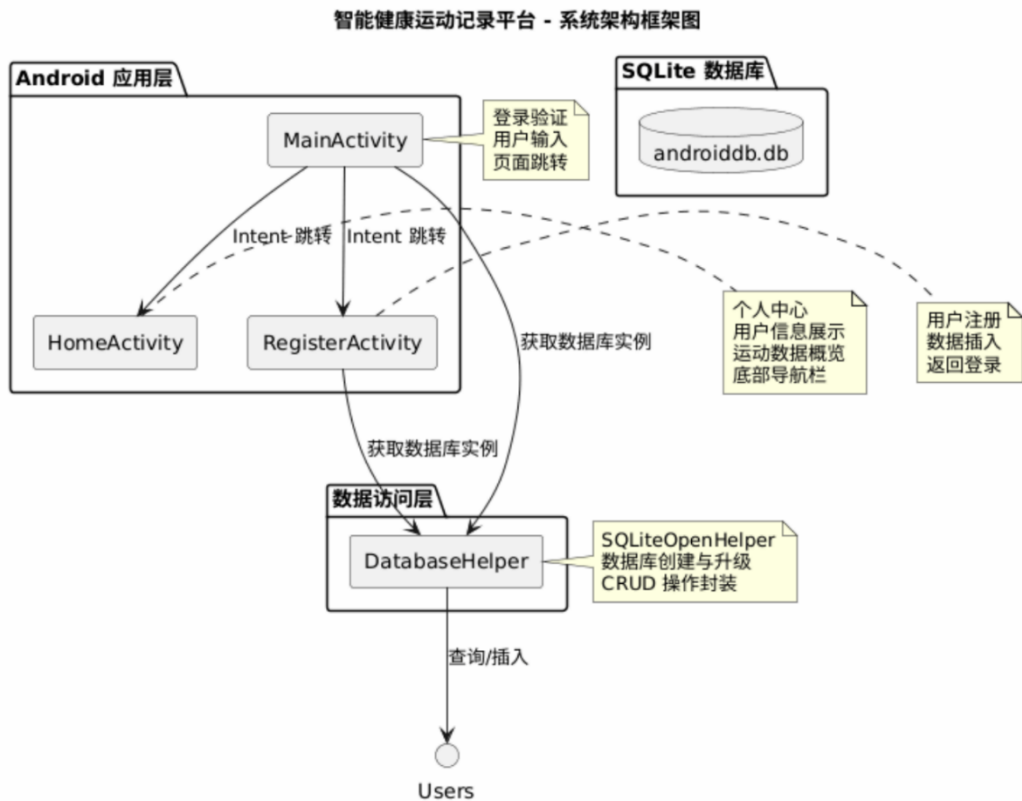


图 1 系统架构图

2 系统类图

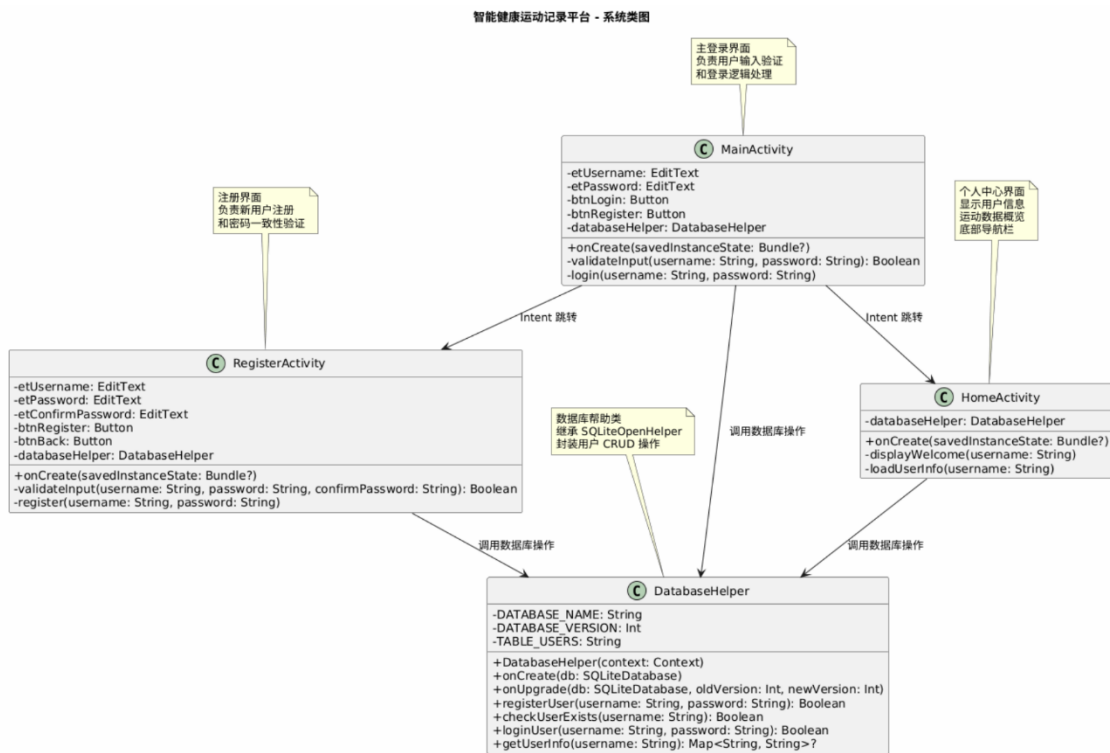


图 2 系统类图

3 系统顺序图

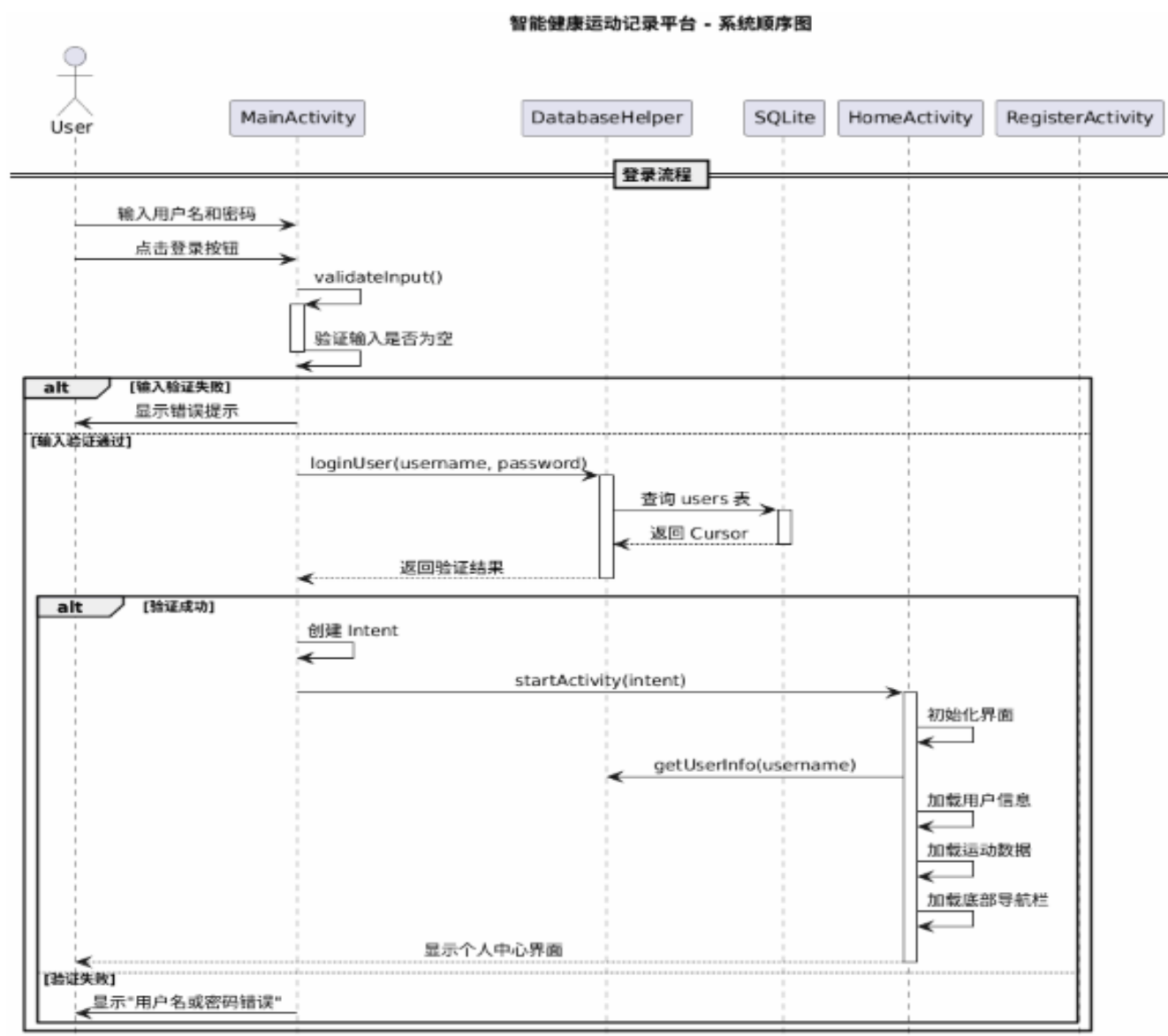


图.3 登录功能顺序图

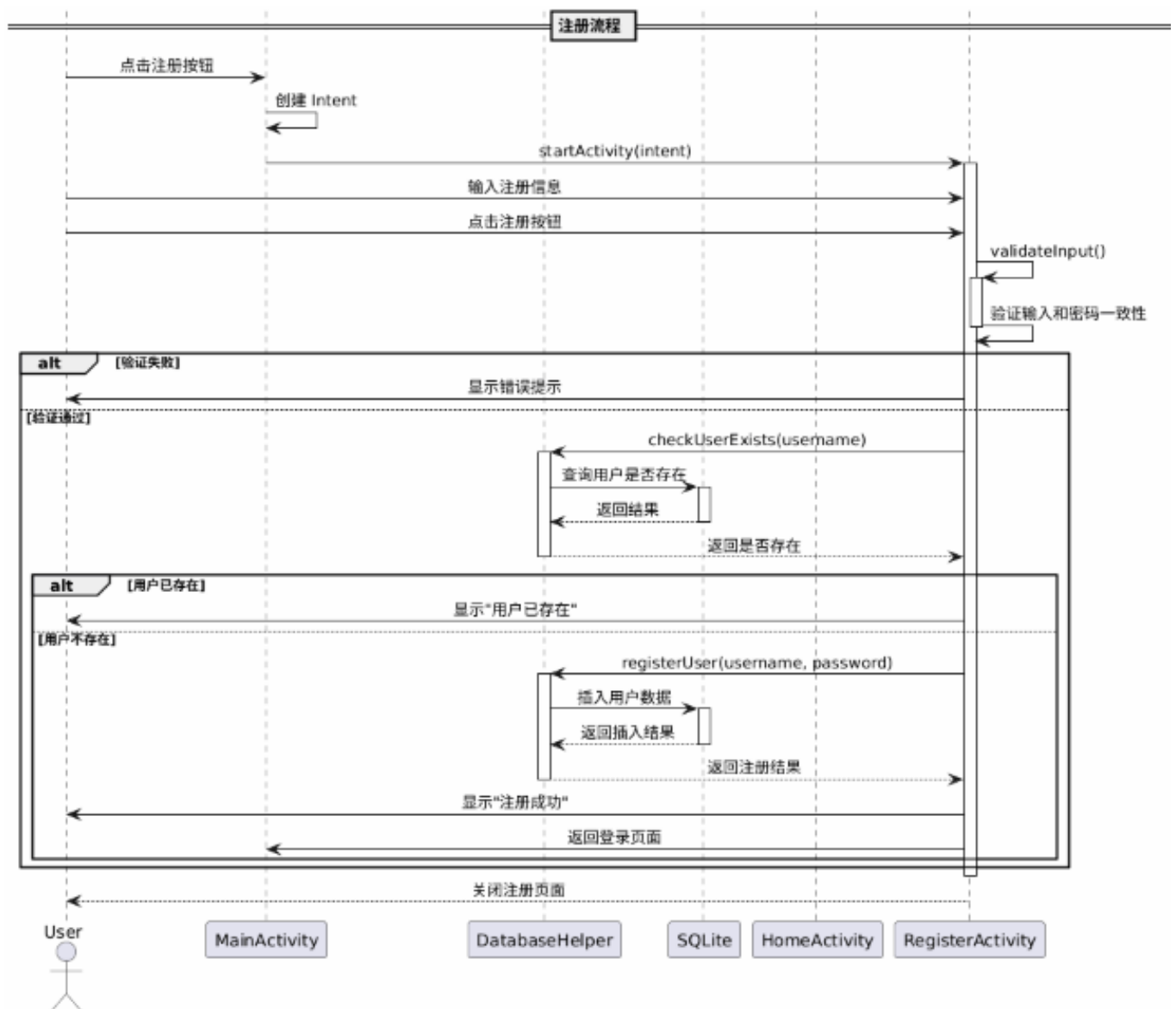


图 4 注册功能顺序图

2.3 布局实现

```
<!-- activity_main.xml -->
```

```
<LinearLayout
```

```
    android:orientation="vertical"
```

```
    android:padding="16dp">
```

```
<EditText
```

```
    android:id="@+id/et_username"
```

```
    android:hint="请输入用户名"
```

```
android:inputType="text"/>
```

```
<EditText
```

```
    android:id="@+id/et_password"
```

```
    android:hint="请输入密码"
```

```
    android:inputType="textPassword"/>
```

```
<Button
```

```
    android:id="@+id/btn_login"
```

```
    android:text="登录"/>
```

```
<Button
```

```
    android:id="@+id/btn_register"
```

```
    android:text="注册"/>
```

```
</LinearLayout>
```

2.4 登录逻辑实现

```
// MainActivity.kt
```

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var etUsername: EditText
```

```
    private lateinit var etPassword: EditText
```

```
    private lateinit var btnLogin: Button
```

```
    private lateinit var btnRegister: Button
```

```
    private lateinit var dbHelper: DatabaseHelper
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_main)
```

```
        etUsername = findViewById(R.id.et_username)
```

```
        etPassword = findViewById(R.id.et_password)
```

```
        btnLogin = findViewById(R.id.btn_login)
```

```
        btnRegister = findViewById(R.id.btn_register)
```

```
        dbHelper = DatabaseHelper(this)
```

```

btnLogin.setOnClickListener {
    val username = etUsername.text.toString()
    val password = etPassword.text.toString()

    if (validateInput(username, password)) {
        login(username, password)
    }
}
btnRegister.setOnClickListener {
    val intent = Intent(this, RegisterActivity::class.java)
    startActivity(intent)
}

private fun validateInput(username: String, password: String): Boolean {
    if (username.isEmpty()) {
        etUsername.error = "用户名不能为空"
        return false
    }
    if (password.isEmpty()) {
        etPassword.error = "密码不能为空"
        return false
    }
    return true
}

private fun login(username: String, password: String) {
    val db = dbHelper.readableDatabase
    val cursor = db.query("users", arrayOf("password"), "username=?",
arrayOf(username), null, null, null)

    if (cursor.moveToFirst()) {
        val storedPassword = cursor.getString(0)
        if (password == storedPassword) {
            // 跳转到个人中心
            val intent = Intent(this, HomeActivity::class.java)
            intent.putExtra("username", username)
            startActivity(intent)
        } else {
            Toast.makeText(this, "用户名或密码错误", Toast.LENGTH_SHORT).show()
        }
    } else {
        Toast.makeText(this, "用户不存在", Toast.LENGTH_SHORT).show()
    }
    cursor.close()
}
}

```

2.5 注册页面实现

```

// RegisterActivity.kt
class RegisterActivity : AppCompatActivity() {

    private lateinit var etUsername: EditText
    private lateinit var etPassword: EditText
    private lateinit var etConfirmPassword: EditText
    private lateinit var btnRegister: Button
    private lateinit var dbHelper: DatabaseHelper

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_register)

        etUsername = findViewById(R.id.et_username)
        etPassword = findViewById(R.id.et_password)
        etConfirmPassword = findViewById(R.id.et_confirm_password)
        btnRegister = findViewById(R.id.btn_register)
        dbHelper = DatabaseHelper(this)

        btnRegister.setOnClickListener {
            val username = etUsername.text.toString()
            val password = etPassword.text.toString()
            val confirmPassword = etConfirmPassword.text.toString()

            if (validateInput(username, password, confirmPassword)) {
                register(username, password)
            }
        }
    }

    private fun validateInput(username: String, password: String, confirmPassword: String):
Boolean {
        if (username.isEmpty()) {
            etUsername.error = "用户名不能为空"
            return false
        }
        if (password.isEmpty()) {
            etPassword.error = "密码不能为空"
            return false
        }
        if (password != confirmPassword) {
            etConfirmPassword.error = "两次输入的密码不一致"
            return false
        }
        return true
    }

    private fun register(username: String, password: String) {
        val db = dbHelper.writableDatabase
    }
}

```



```

        val cursor = db.query("users", arrayOf("id"), "username=?", arrayOf(username), null,
null, null)

        if (cursor.moveToFirst()) {
            Toast.makeText(this, "用户已存在", Toast.LENGTH_SHORT).show()
        } else {
            val values = ContentValues()
            values.put("username", username)
            values.put("password", password)
            values.put("register_time", System.currentTimeMillis())
            db.insert("users", null, values)
            Toast.makeText(this, "注册成功", Toast.LENGTH_SHORT).show()
            finish()
        }
        cursor.close()
    }
}

```

2.6 个人中心页面

```

// HomeActivity.kt
class HomeActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_home)

        val username = intent.getStringExtra("username")
        val tvWelcome = findViewById<TextView>(R.id.tv_welcome)
        tvWelcome.text = "欢迎 $username"
    }
}

```

2.7 数据库辅助类

```

// DatabaseHelper.kt
class DatabaseHelper(context: Context) : SQLiteOpenHelper(context, "androiddb.db",
null, 1) {

    override fun onCreate(db: SQLiteDatabase) {
        val createTable = "CREATE TABLE users (id INTEGER PRIMARY KEY
AUTOINCREMENT, username TEXT UNIQUE, password TEXT, register_time
INTEGER)"
        db.execSQL(createTable)
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {

```

```
        db.execSQL("DROP TABLE IF EXISTS users")
        onCreate(db)
    }
}
```

实验结果

验证结果

平台	登录功能	输入验证	页面跳转	数据传递
Android	[√] 成功	[√] 成功	[√] 成功	[√] 成功

表 2 实验验证结果

截图记录

Android 登录页面

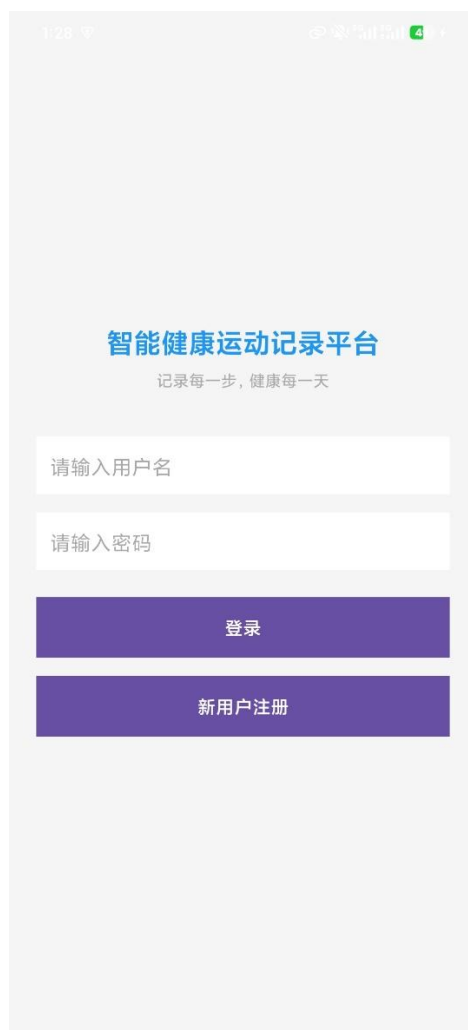


图 5 登录页面真机测试截图

Android 个人中心



图 6 个人中心真机测试截图

Android 注册页面

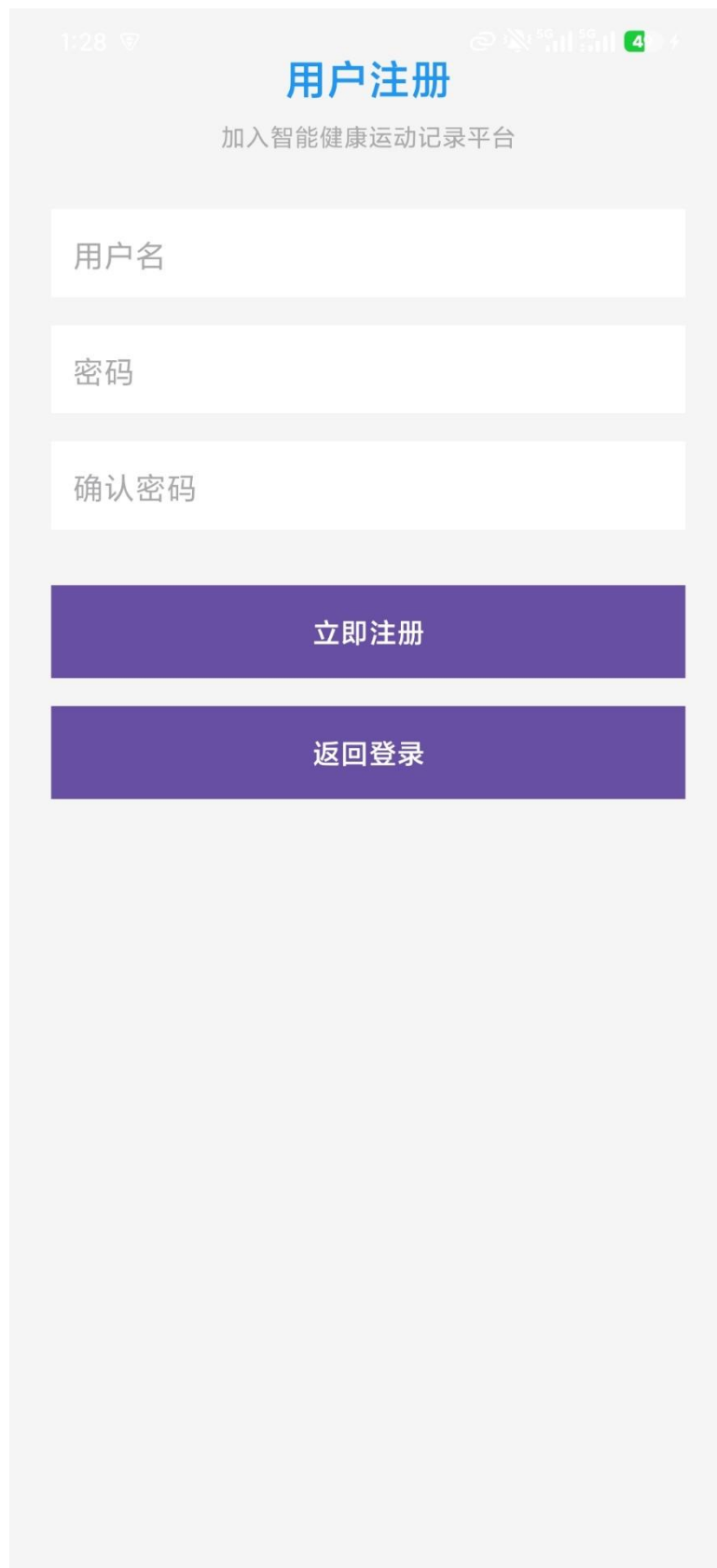


图 7 注册页面真机测试截图

平台差异对比

对比项	Android	iOS
开发语言	Kotlin	Swift
UI 框架	XML	Swift UI
页面跳转	Intent	NavigationLink
数据传递	Intent.putExtra	@Binding
布局方式	线性布局/约束布局	Stack/Grid
生命周期	Activity	ViewController

表 3 平台差异对比表

问题与解决

问题描述	解决方案	解决结果
数据库创建失败	确保 SQLiteOpenHelper 正确实现	数据库创建成功
密码验证失败	检查数据库查询逻辑	密码验证正常
注册时用户已存在	添加用户存在性检查	注册逻辑正确

表 4 实验中的问题与解决方法

实验总结

实验收获

- 掌握 Android Kotlin 登录页面开发
- 学会 Activity 间数据传递
- 掌握输入验证逻辑实现
- 理解 SQLite 数据库连接和操作的基本流程
- 学会使用 Android 内置 SQLite 数据库存储用户信息
- 掌握个人中心页面设计和底部导航栏实现
- 学会卡片式布局和 Material Design 风格应用

技术要点

- **Kotlin 语言特性**：使用 lateinit var 延迟初始化、companion object 定义常量
- **Activity 生命周期**：onCreate 方法的使用、Intent 跳转
- **UI 控件使用**：EditText 输入框、Button 按钮、Toast 提示
- **数据库操作**：SQLiteOpenHelper、ContentValues、Cursor 查询
- **输入验证**：空值检查、密码一致性验证
- **布局设计**：LinearLayout 嵌套、卡片式布局、底部导航栏

考核指标

考核项	评分标准	得分
功能完整性	登录功能完整，输入验证正确	25 分
代码质量	代码结构清晰，注释完整	20 分
页面跳转	Activity/View 跳转正常	15 分
数据传递	数据传递正确	15 分
平台对比	完成平台差异分析	15 分
实验报告	报告结构完整，内容详实	10 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名： _____

日期： _____